

Exercise 1: Implementing the Singleton Pattern

Solution

The **Singleton Design Pattern** ensures that only one object of a class exists throughout the application's execution. This is useful for shared resources such as logging, configuration management, and database connections.

```
class LogManager {

    private static LogManager loggerInstance;

    private LogManager() {
        System.out.println("Logger instance initialized.");
    }

    public static LogManager getInstance() {

        if (loggerInstance == null) {
            loggerInstance = new LogManager();
        }

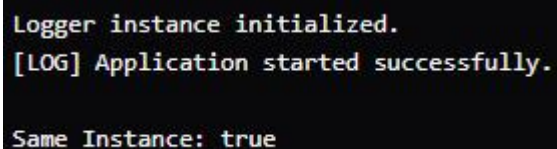
        return loggerInstance;
    }

    public void logMessage(String message) {
        System.out.println("[LOG] " + message);
    }
}

public class SingletonDemo {
```

```
public static void main(String[] args) {  
  
    LogManager firstLogger = LogManager.getInstance();  
    LogManager secondLogger = LogManager.getInstance();  
  
    firstLogger.logMessage("Application started successfully.");  
  
    System.out.println("\nSame Instance: " + (firstLogger == secondLogger));  
}  
}
```

Output:

A screenshot of a terminal window with a black background and white text. The output shows three lines: 'Logger instance initialized.', '[LOG] Application started successfully.', and 'Same Instance: true'.

```
Logger instance initialized.  
[LOG] Application started successfully.  
  
Same Instance: true
```

Analysis:

The Singleton pattern guarantees that only one object of the logging class is created during the application's lifetime. Every request for the logger returns the same instance, ensuring consistent logging while reducing unnecessary object creation.

Exercise 2: Implementing the Factory Method Pattern

Solution

The **Factory Method Pattern** provides an interface for creating objects while allowing subclasses to determine the specific object to instantiate. This improves flexibility and reduces dependency on concrete classes.

```
public class FactoryMethodDemo {

    public static void main(String[] args) {

        FileFactory wordCreator = new WordFileFactory();
        FileFactory pdfCreator = new PdfFileFactory();
        FileFactory excelCreator = new ExcelFileFactory();

        FileDocument word = wordCreator.createFile();
        FileDocument pdf = pdfCreator.createFile();
        FileDocument excel = excelCreator.createFile();

        word.display();
        pdf.display();
        excel.display();
    }
}

interface FileDocument {

    void display();
}

class WordFile implements FileDocument {

    @Override
    public void display() {
```

```
        System.out.println("Word document is opened.");
    }
}
```

```
class PdfFile implements FileDocument {
```

```
    @Override
    public void display() {
        System.out.println("PDF document is opened.");
    }
}
```

```
class ExcelFile implements FileDocument {
```

```
    @Override
    public void display() {
        System.out.println("Excel spreadsheet is opened.");
    }
}
```

```
abstract class FileFactory {
```

```
    public abstract FileDocument createFile();
}
```

```
class WordFileFactory extends FileFactory {
```

```
    @Override
    public FileDocument createFile() {
        return new WordFile();
    }
}
```

```
class PdfFileFactory extends FileFactory {
```

```
    @Override
```

```
    public FileDocument createFile() {
```

```
        return new PdfFile();
```

```
    }
```

```
}
```

```
class ExcelFileFactory extends FileFactory {
```

```
    @Override
```

```
    public FileDocument createFile() {
```

```
        return new ExcelFile();
```

```
    }
```

```
}
```

Output:

```
Word document is opened.  
PDF document is opened.  
Excel spreadsheet is opened.
```

Analysis:

The Factory Method pattern separates object creation from object usage. Instead of creating objects directly using constructors, the client requests them through factory classes. This approach improves maintainability, makes the code easier to extend with new document types, and follows the Open/Closed Principle by allowing new factories to be added without modifying existing client code.